

Chapter 11: Duality

Algorithms for Optimization, 2nd ed. (2026)

Kochenderfer & Wheeler

MIT Press

Outline

- 1 11.1 Dual Problem
- 2 11.2 Primal-Dual Methods
- 3 11.3 Dual Ascent
- 4 11.4 ADMM
- 5 11.5 ADMM Applications
- 6 11.6 Distributed Methods
- 7 Summary & Exercises

Motivation: Why Duality? I

Most optimisation problems start as a *primal* problem: minimise some objective subject to constraints. Duality is a powerful theoretical tool that lets us construct a companion *dual* problem whose solution always provides a guaranteed lower bound on the primal optimal value.

The key benefits of working with duality are:

(1) Verification: when the dual optimal value equals the primal optimal value — a situation called *strong duality* — solving either problem gives us the answer to both, and we can certify optimality without exhaustive search.

(2) Algorithm design: many of the most efficient practical solvers — interior-point methods, ADMM (Alternating Direction Method of Multipliers), and distributed optimisation algorithms — are built directly on the dual structure of the problem.

(3) Sensitivity analysis: the dual variables (also called Lagrange multipliers, explained below) tell us how much the optimal objective value would change if we tightened or relaxed each constraint by a small amount.

Lagrangian Duality Framework

Primal: $\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{s.t. } g_i(\mathbf{x}) \leq 0, \quad h_j(\mathbf{x}) = 0$

Lagrangian: $\mathcal{L}(\mathbf{x}, \mu, \lambda) = f(\mathbf{x}) + \sum_i \mu_i g_i(\mathbf{x}) + \sum_j \lambda_j h_j(\mathbf{x})$

Dual function: $D(\mu, \lambda) = \min_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \mu, \lambda)$

Dual problem: $\max_{\mu \geq 0, \lambda} D(\mu, \lambda)$

Weak duality: $d^* \leq p^*$ Strong duality: $d^* = p^*$ (convex + CQ)

Motivation: Why Duality? II

Key Insight

The dual problem is always a *concave maximisation* problem, regardless of whether the primal is convex. This means dual ascent algorithms always converge — a very useful property.

The Generalised Lagrangian I

We begin with the most general form of a constrained optimisation problem. Suppose we want to minimise an objective $f(\mathbf{x})$ subject to inequality constraints $\mathbf{g}(\mathbf{x}) \leq \mathbf{0}$ and equality constraints $\mathbf{h}(\mathbf{x}) = \mathbf{0}$:

General Constrained Problem

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{subject to} \quad \mathbf{g}(\mathbf{x}) \leq \mathbf{0}, \quad \mathbf{h}(\mathbf{x}) = \mathbf{0}$$

where $\mathbf{x} \in \mathbb{R}^n$ ($\mathbf{x}$, an n -dimensional real vector) is the vector of decision variables we are optimising over.

The central idea of duality is to “relax” the constraints by moving them into the objective function, weighted by penalty multipliers. The result is the **Generalised Lagrangian** (named after the mathematician Joseph-Louis Lagrange):

Generalised Lagrangian

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\mu}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \sum_i \mu_i g_i(\mathbf{x}) + \sum_j \lambda_j h_j(\mathbf{x})$$

The Generalised Lagrangian II

Notation

- $\mathbf{x} \in \mathbb{R}^n$: the *primal* (design) variables — the quantities we are trying to find.
- $\boldsymbol{\mu} \geq \mathbf{0}$ (mu, pronounced “myoo”, a bold Greek letter): a vector of *inequality multipliers*, one per inequality constraint $g_i(\mathbf{x}) \leq 0$. Each $\mu_i \geq 0$ must be non-negative.
- $\boldsymbol{\lambda} \in \mathbb{R}^p$ (lambda, pronounced “lam-da”, a bold Greek letter): a vector of *equality multipliers*, one per equality constraint $h_j(\mathbf{x}) = 0$. These can be any sign.
- $g_i(\mathbf{x})$: the i -th inequality constraint function evaluated at $\mathbf{x}$; the constraint requires this to be ≤ 0 .
- $h_j(\mathbf{x})$: the j -th equality constraint function; the constraint requires this to equal 0 exactly.
- $\sum_i$ (Sigma, meaning “sum over index i ”): we add up one term for each inequality constraint.

The Generalised Lagrangian III

How to Read This Formula

Think of the Lagrangian as an “augmented objective.” When $\mathbf{x}$ satisfies all inequality constraints, $g_i(\mathbf{x}) \leq 0$, and since $\mu_i \geq 0$, the penalty terms $\mu_i g_i(\mathbf{x})$ are non-positive — they do not inflate the objective. But if $\mathbf{x}$ violates a constraint (so $g_i(\mathbf{x}) > 0$), the penalty $\mu_i g_i(\mathbf{x})$ becomes *positive*, pushing the Lagrangian above $f(\mathbf{x})$ and discouraging infeasibility. The multipliers μ_i and λ_j are also called **dual variables** because they become the variables of the dual problem.

Primal and Dual Problems I

Starting from the Lagrangian $\mathcal{L}(\mathbf{x}, \boldsymbol{\mu}, \boldsymbol{\lambda})$, we can recover the original constrained problem by a clever minimax trick, and then derive the dual by swapping the order of min and max.

Primal Problem

$$p^* = \min_{\mathbf{x}} \max_{\boldsymbol{\mu} \geq 0, \boldsymbol{\lambda}} \mathcal{L}(\mathbf{x}, \boldsymbol{\mu}, \boldsymbol{\lambda})$$

Here p^* (p-star) is the *optimal primal value* — the lowest possible value of $f(\mathbf{x})$ subject to all constraints.

Why does this recover the original problem? If $\mathbf{x}$ violates any inequality constraint, then $g_i(\mathbf{x}) > 0$ for some i , and we can send $\mu_i \rightarrow +\infty$, making the inner maximum $+\infty$. An outer minimisation over $\mathbf{x}$ will never choose an infeasible point. For feasible $\mathbf{x}$, $g_i(\mathbf{x}) \leq 0$, the best multipliers set $\mu_i = 0$ and $\lambda_j = 0$, so the maximum equals $f(\mathbf{x})$.

Primal and Dual Problems II

Dual Function

$$\mathcal{D}(\boldsymbol{\mu}, \boldsymbol{\lambda}) = \min_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\mu}, \boldsymbol{\lambda})$$

For fixed multipliers $(\boldsymbol{\mu}, \boldsymbol{\lambda})$, this is the *minimum value of the Lagrangian over all $\mathbf{x}$* (with no constraints on $\mathbf{x}$).

The dual function $\mathcal{D}$ (script D) has a remarkable property: it is always **concave** in $(\boldsymbol{\mu}, \boldsymbol{\lambda})$, even when the original problem is non-convex. This is because it is a pointwise minimum of affine (linear) functions of $(\boldsymbol{\mu}, \boldsymbol{\lambda})$, and a minimum of affines is always concave.

Dual Problem

$$d^* = \max_{\boldsymbol{\mu} \geq \mathbf{0}, \boldsymbol{\lambda}} \mathcal{D}(\boldsymbol{\mu}, \boldsymbol{\lambda})$$

Here d^* (d-star) is the *optimal dual value* — the highest value of the dual function achievable over all valid multipliers.

Primal and Dual Problems III

Key Insight

We have transformed a minimisation problem into a maximisation problem. The dual problem is always concave, which makes it easier to solve with gradient ascent methods — they are guaranteed to find the global maximum of a concave function.

Weak and Strong Duality I

Weak Duality (always true)

$$d^* \leq p^*$$

The dual optimal value is *always* a lower bound on the primal optimal value. This holds regardless of convexity or any other assumption.

The difference $p^* - d^* \geq 0$ is called the **duality gap**. A large gap means the dual bound is loose; a gap of zero means we have found exact optimality certificates.

Strong Duality

$$d^* = p^*$$

The gap is zero — the dual and primal optimal values coincide. Strong duality holds for **convex** problems that satisfy a *constraint qualification*, such as:

Weak vs. Strong Duality

duality gap
↔

Strong duality:
 $d^* = p^*$

d^* (dual) p^* (primal)

Weak and Strong Duality II

How to Read This

Think of the primal value p^* as the “true answer” and the dual value d^* as a “certificate” that no solution can do better. Weak duality says the certificate can never exceed the truth. Strong duality says the certificate is tight — we can verify optimality exactly. In practice, algorithms drive the duality gap to near-zero as a stopping criterion.

Example 11.1: Dual Function as Lower Bound I

This example makes the abstract notion of “lower bound” concrete.

The problem: minimise $\sin(x)$ subject to $x^2 \leq 1$ (so x must lie in the interval $[-1, 1]$).

The minimum of $\sin(x)$ on $[-1, 1]$ occurs at $x^* = -1$, giving $p^* = \sin(-1) \approx -0.841$.

Constructing the Lagrangian: the single inequality constraint is $g(x) = x^2 - 1 \leq 0$, so the Lagrangian is:

$$\mathcal{L}(x, \mu) = \sin(x) + \mu(x^2 - 1)$$

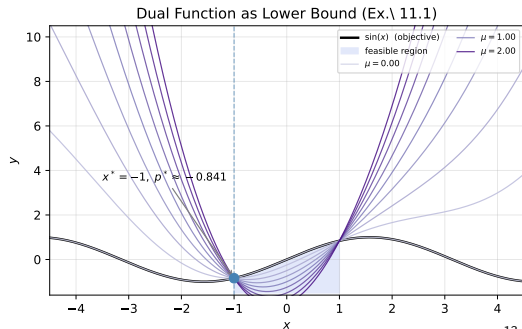
where $\mu \geq 0$ (μ) is the scalar inequality multiplier.

Primal (minimax):

$$p^* = \min_x \max_{\mu \geq 0} [\sin(x) + \mu(x^2 - 1)]$$

Dual (maximin):

$$d^* = \max_{\mu} \min_x [\sin(x) + \mu(x^2 - 1)]$$



Example 11.2: Lagrangian Duality with Equality Constraint I

Here we work through a two-variable problem with an equality constraint, deriving the dual function explicitly and verifying strong duality.

The problem:

$$\min_{\mathbf{x}} x_1 + x_2 + x_1 x_2 \quad \text{subject to} \quad x_1^2 + x_2^2 = 1$$

The constraint forces (x_1, x_2) to lie on the unit circle (the set of points at distance 1 from the origin).

Constructing the Lagrangian: the single equality constraint is $h(x_1, x_2) = x_1^2 + x_2^2 - 1 = 0$, so:

$$\mathcal{L}(x_1, x_2, \lambda) = x_1 + x_2 + x_1 x_2 + \lambda (x_1^2 + x_2^2 - 1)$$

where λ (lambda, pronounced “lam-da”) is the equality multiplier (unrestricted in sign).

Finding critical points by setting partial derivatives to zero:

$$\frac{\partial \mathcal{L}}{\partial x_1} = 1 + x_2 + 2\lambda x_1 = 0, \quad \frac{\partial \mathcal{L}}{\partial x_2} = 1 + x_1 + 2\lambda x_2 = 0, \quad x_1^2 + x_2^2 = 1$$

Here $\frac{\partial \mathcal{L}}{\partial x_1}$ means the partial derivative of $\mathcal{L}$ with respect to x_1 , i.e. the rate of change of $\mathcal{L}$ as x_1 varies while x_2 and λ are held fixed.

Example 11.2: Lagrangian Duality with Equality Constraint II

Solving the three equations simultaneously yields **four critical points**:

x_1	x_2	λ	Objective $f(\mathbf{x})$
-1	0	1/2	-1 (global minimum)
0	-1	1/2	-1 (global minimum)
$\frac{\sqrt{2}+1}{\sqrt{2}+2}$	$\frac{\sqrt{2}+1}{\sqrt{2}+2}$	$\frac{1}{2}(-1-\sqrt{2})$	≈ 1.914 (global maximum)
$\frac{\sqrt{2}-1}{\sqrt{2}-2}$	$\frac{\sqrt{2}-1}{\sqrt{2}-2}$	$\frac{1}{2}(-1+\sqrt{2})$	≈ -0.914 (saddle point)

The optimal primal value is $p^* = -1$, achieved at $(-1, 0)$ and $(0, -1)$.

Example 11.2: Lagrangian Duality with Equality Constraint III

Deriving the dual function: substituting $u = x_1 + x_2$ and $v = x_1 - x_2$ transforms the Lagrangian into:

$$\mathcal{L} = \frac{1}{4}(2\lambda + 1)u^2 + u + \frac{1}{4}(2\lambda - 1)v^2 - \lambda$$

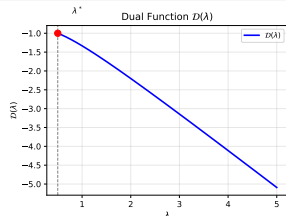
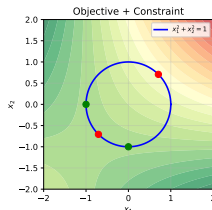
Minimising over u and v gives the dual function:

Dual Function $\mathcal{D}(\lambda)$

$$\mathcal{D}(\lambda) = \begin{cases} -\left(\frac{1}{2\lambda + 1} + \lambda\right) & \lambda \geq \frac{1}{2} \\ -\infty & \text{otherwise} \end{cases}$$

The dual function equals $-\infty$ for $\lambda < 1/2$ because the quadratic in v becomes unbounded below. For $\lambda \geq 1/2$, we can minimise the Lagrangian, and the maximum of $\mathcal{D}(\lambda)$ occurs at $\lambda^* = 1/2$, giving $d^* = -(1/2 + 1/2) = -1 = p^*$.

This confirms **strong duality**: $d^* = p^* = -1$.



Primal-Dual Interior-Point: Motivation I

Primal-dual interior-point methods are among the most powerful algorithms for solving convex constrained optimisation problems. The idea is to update the primal variables $\mathbf{x}$ and the dual variables μ (mu) *simultaneously* at each iteration, rather than alternating between them. This joint update leads to very fast convergence in practice.

We focus on the inequality-constrained problem:

Inequality-Constrained Problem

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{subject to} \quad \mathbf{g}(\mathbf{x}) \leq \mathbf{0}$$

where $\mathbf{g}(\mathbf{x}) = [g_1(\mathbf{x}), \dots, g_m(\mathbf{x})]^\top$ is the vector of m inequality constraint functions.

Step 1 — Replace constraints with a log-barrier: instead of enforcing $g_i(\mathbf{x}) \leq 0$ directly, we add a *logarithmic barrier* that blows up as $\mathbf{x}$ approaches the constraint boundary:

Primal-Dual Interior-Point: Motivation II

Log-Barrier Problem

$$\min_{\mathbf{x}} f(\mathbf{x}) - \frac{1}{\rho} \sum_{i=1}^m \log(-g_i(\mathbf{x}))$$

Notation

- ρ (rho, pronounced “roe”): a positive penalty parameter that controls how strongly the barrier enforces constraints. As $\rho \rightarrow \infty$, the barrier becomes more and more concentrated at the boundary.
- $\log(\cdot)$: the natural logarithm. When $g_i(\mathbf{x})$ is negative (feasible), $-g_i(\mathbf{x})$ is positive, so $\log(-g_i(\mathbf{x}))$ is finite. As $g_i(\mathbf{x}) \rightarrow 0$ from below, $-g_i(\mathbf{x}) \rightarrow 0^+$ and $\log(-g_i(\mathbf{x})) \rightarrow -\infty$, creating a barrier wall at the boundary.
- $\frac{1}{\rho}$: the barrier weight. A small weight (large ρ) means the barrier is almost invisible far from the boundary, so the solution stays close to the true constrained optimum.

Primal-Dual Interior-Point: Motivation III

Step 2 — Interpret the optimality condition: setting the gradient of the barrier objective to zero gives:

$$\mathbf{0} = \nabla f(\mathbf{x}^*) + \sum_{i=1}^m \underbrace{\frac{-1}{\rho g_i(\mathbf{x}^*)}}_{\mu_i^*} \nabla g_i(\mathbf{x}^*)$$

Here ∇ (nabla, the del or gradient operator) denotes the vector of partial derivatives. We can define $\mu_i^* = -1/(\rho g_i(\mathbf{x}^*))$, which is automatically positive since $g_i(\mathbf{x}^*) < 0$ for a strictly interior point. This recovers the KKT stationarity condition:

$$\mathbf{0} = \nabla f(\mathbf{x}^*) + \sum_{i=1}^m \mu_i^* \nabla g_i(\mathbf{x}^*)$$

Step 3 — Duality gap: one can show that the duality gap at the barrier solution equals m/ρ , where m is the number of constraints. As $\rho \rightarrow \infty$, the gap shrinks to zero, confirming convergence to the true constrained optimum.

Primal-Dual Interior-Point: Motivation IV

Key Insight

By increasing ρ gradually, the log-barrier solution tracks the true constrained optimum ever more closely. Primal-dual methods do not solve a sequence of separate barrier problems; instead, they use a Newton step that simultaneously drives both the primal residual and the duality gap to zero.

Primal-Dual System and Newton Step I

The primal-dual method seeks to satisfy both *stationarity* and *complementary slackness* simultaneously. It does this by defining a residual vector $\mathbf{r}$ that packages all the optimality conditions, and then applying a Newton step to drive $\mathbf{r}$ to zero.

Residual Vector $\mathbf{r}(\mathbf{x}, \boldsymbol{\mu}, \rho)$

$$\mathbf{r}(\mathbf{x}, \boldsymbol{\mu}, \rho) = \begin{bmatrix} \nabla f(\mathbf{x}) + \sum_i \mu_i \nabla g_i(\mathbf{x}) \\ -\mu_1 g_1(\mathbf{x}) - 1/\rho \\ \vdots \\ -\mu_m g_m(\mathbf{x}) - 1/\rho \end{bmatrix} = \mathbf{0}$$

Primal-Dual System and Newton Step II

Notation

- The first block (a vector of length n) is the KKT stationarity condition: the gradient of the Lagrangian with respect to $\mathbf{x}$ must vanish at the optimum.
- The remaining m scalar equations are the *perturbed complementary slackness* conditions: at a true KKT point we would want $\mu_i g_i(\mathbf{x}) = 0$, but the barrier shifts this to $-\mu_i g_i(\mathbf{x}) = 1/\rho$. As $\rho \rightarrow \infty$, the right side shrinks to zero, recovering exact complementary slackness.

Primal-Dual System and Newton Step III

Newton (Descent) Direction

$$\mathbf{d} = -\mathbf{J}^{-1} \mathbf{r}$$

where $\mathbf{J} = \partial \mathbf{r} / \partial (\mathbf{x}, \boldsymbol{\mu})$ is the Jacobian (matrix of partial derivatives) of the residual with respect to all unknowns $(\mathbf{x}, \boldsymbol{\mu})$.

Notation

- $\mathbf{J}^{-1}$: the inverse of the Jacobian matrix. Solving the linear system $\mathbf{J} \mathbf{d} = -\mathbf{r}$ gives the Newton direction without explicitly forming the inverse.
- $\mathbf{d} = [d_x; d_\mu]$: partitioned into the primal update direction $d_x \in \mathbb{R}^n$ and the dual update direction $d_\mu \in \mathbb{R}^m$.

Step-size safeguards: a plain Newton step might violate the requirements $\boldsymbol{\mu} > \mathbf{0}$ (dual positivity) or $\mathbf{g}(\mathbf{x}) \leq \mathbf{0}$ (primal feasibility). Three line-search rules prevent this:

Primal-Dual System and Newton Step IV

- ① **Dual positivity:** if the i -th component of d_μ is negative (i.e. $d_{\mu_i} < 0$, the step would decrease μ_i), limit the step size to $\alpha \leq (1 - \varepsilon) \mu_i / (-d_{\mu_i})$ so that μ_i stays positive. Here ε (epsilon, a small positive constant, e.g. 0.01) provides a safety margin.
- ② **Primal feasibility:** repeatedly halve α (multiply by $p < 1$ each time) until $g_i(\mathbf{x} + \alpha d_x) \leq 0$ for all i , i.e. the new primal point remains feasible.
- ③ **Armijo sufficient decrease:** reduce α further until $\|\mathbf{r}(\mathbf{x} + \alpha d_x, \boldsymbol{\mu} + \alpha d_\mu, \rho)\| \leq (1 - \beta\alpha) \|\mathbf{r}\|$, ensuring the residual norm decreases by at least a fraction $\beta\alpha$ of its current value. Here β (beta, a small constant, e.g. 10^{-4}) controls how strict the decrease requirement is.

Example 11.3: Primal-Dual Descent Direction I

We now trace through the primal-dual algorithm on a concrete scalar example to see exactly what the residual and Jacobian look like.

The problem: minimise $f(x) = x^2$ subject to $(x - 3)^2 \leq 1$, i.e. x must lie in $[2, 4]$.

The constraint function is $g(x) = (x - 3)^2 - 1$. The Lagrangian is:

$$\mathcal{L}(x, \mu) = x^2 + \mu [(x - 3)^2 - 1]$$

Residual vector:

$$\mathbf{r}(x, \mu, \rho) = \begin{bmatrix} 2x + 2\mu(x - 3) \\ -\mu(x - 3)^2 + \mu - 1/\rho \end{bmatrix} = \mathbf{0}$$

The first equation is stationarity ($\partial\mathcal{L}/\partial x = 0$). The second is perturbed complementary slackness ($-\mu g(x) = 1/\rho$).

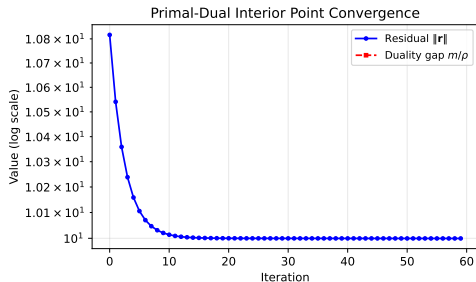
Jacobian: differentiating $\mathbf{r}$ with respect to (x, μ) :

$$\mathbf{J}(x, \mu) = \begin{bmatrix} 2 + 2\mu & 2(x - 3) \\ -2\mu(x - 3) & 1 - (x - 3)^2 \end{bmatrix}$$

Example 11.3: Primal-Dual Descent Direction II

Newton direction: $\mathbf{d} = -\mathbf{J}^{-1}\mathbf{r}$, where d_1 (d sub 1) updates x and d_2 (d sub 2) updates μ .

Starting from a feasible initial point (e.g. $x = 2.5$, $\mu = 1$), the algorithm applies the Newton direction with step-size safeguards. Because Newton's method has quadratic convergence near the solution, the algorithm reaches the optimum $x^* = 2$, $\mu^* = 1/4$ in very few iterations. The figure shows the rapid reduction of the residual norm and the duality gap.



Python: Primal-Dual Interior-Point (Algorithm 11.2)

```
1 import numpy as np
2 from scipy.linalg import solve
3
4 def primal_dual_interior_point(f, gradf, hessf, gs, gradgs, hessgs,
5                               rho, x, mu, alpha_max=1.0, beta=1e-4,
6                               gamma=2.0, eps=0.01, p=0.9,
7                               res_min=1e-4, gap_min=1e-4):
8     """
9     Primal-dual interior point method for:
10     minimize f(x)    subject to g_i(x) <= 0    for i in 1..m
11     """
12     m = len(gs)
13     def residual(x, mu, rho):
14         r_x = gradf(x) + sum(mu[i]*gradgs[i](x) for i in range(m))
15         r_mu = [-mu[i]*gs[i](x) - 1/rho for i in range(m)]
16         return np.concatenate([r_x, r_mu])
17
18     res = np.linalg.norm(residual(x, mu, rho))
19     gap = sum(-mu[i]*gs[i](x) for i in range(m))
20
21     while res > res_min or gap > gap_min:
22         rho *= gamma
23         # Build Jacobian and solve Newton system
24         n = len(x)
25         J = np.zeros((n+m, n+m))
26         J[:n, :n] = hessf(x) + sum(mu[i]*hessgs[i](x) for i in range(m))
27         for i in range(m):
```

Dual Ascent: Alternating Optimisation I

Dual ascent is an iterative method that alternates between minimising the Lagrangian over the primal variables $\mathbf{x}$ and ascending (maximising) the dual function over the multipliers $\boldsymbol{\mu}$ and $\boldsymbol{\lambda}$ (lambda). Because the dual function is concave, gradient ascent on it converges to the global maximum — the dual optimal solution.

Why does this work? When strong duality holds, the optimal triple $(\mathbf{x}^*, \boldsymbol{\mu}^*, \boldsymbol{\lambda}^*)$ is a saddle point of the Lagrangian:

$$\mathcal{L}(\mathbf{x}^*, \boldsymbol{\mu}^*, \boldsymbol{\lambda}^*) = \min_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\mu}^*, \boldsymbol{\lambda}^*) \quad \text{and} \quad \mathcal{L}(\mathbf{x}^*, \boldsymbol{\mu}^*, \boldsymbol{\lambda}^*) = \max_{\boldsymbol{\mu} \geq 0, \boldsymbol{\lambda}} \mathcal{L}(\mathbf{x}^*, \boldsymbol{\mu}, \boldsymbol{\lambda})$$

At this saddle point, no change to $\mathbf{x}$ can decrease the Lagrangian, and no change to the multipliers can increase it. Alternating between the two optimisations converges to this point.

Dual Ascent: Alternating Optimisation II

Dual Ascent Algorithm

Step 1 — Primal update (minimise Lagrangian over $\mathbf{x}$):

$$\mathbf{x}^{(k+1)} = \arg \min_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\mu}^{(k)}, \boldsymbol{\lambda}^{(k)})$$

This is an unconstrained minimisation (no constraints on $\mathbf{x}$), which is generally easier to solve than the original constrained problem.

Step 2 — Dual update (gradient ascent on dual variables):

$$\boldsymbol{\mu}^{(k+1)} = \max\left(\mathbf{0}, \boldsymbol{\mu}^{(k)} + \alpha \mathbf{g}(\mathbf{x}^{(k+1)})\right)$$

$$\boldsymbol{\lambda}^{(k+1)} = \boldsymbol{\lambda}^{(k)} + \alpha \mathbf{h}(\mathbf{x}^{(k+1)})$$

Dual Ascent: Alternating Optimisation III

Notation

- $\mathbf{x}^{(k)}$ (x superscript k): the value of $\mathbf{x}$ at iteration number k . Superscript k is a counter, not a power.
- α (alpha, pronounced “al-fa”): the step size, i.e. how far we move along the gradient direction. A larger α means bigger steps; too large causes divergence.
- $\mathbf{g}(\mathbf{x}^{(k+1)})$: the vector of constraint violations at the new primal point. This is the gradient of the dual function $\mathcal{D}$ with respect to $\boldsymbol{\mu}$ — ascent on $\mathcal{D}$ means stepping toward positive constraint violations.
- $\max(\mathbf{0}, \cdot)$: the component-wise maximum with zero, which projects $\boldsymbol{\mu}$ onto the non-negative orthant and ensures $\boldsymbol{\mu} \geq \mathbf{0}$ is maintained.

Dual Ascent: Alternating Optimisation IV

How to Read This

The dual update for μ says: if the constraint $g_i(\mathbf{x}) > 0$ is violated at the new primal point, increase μ_i (make the penalty for violating this constraint bigger). If $g_i(\mathbf{x}) < 0$ (constraint is satisfied with slack), decrease μ_i toward zero. Over many iterations, μ_i settles at the value that perfectly prices the constraint. One drawback is that dual ascent can be slow when the problem is ill-conditioned, because it zig-zags in the dual space.

Dual Ascent: Numerical Example I

The problem: minimise $x_1^2 + x_2^2$ subject to $x_1 + x_2 = 1$. The objective is the squared distance from the origin; the constraint forces the solution to lie on the line $x_1 + x_2 = 1$.

Deriving the dual function: the Lagrangian is $\mathcal{L} = x_1^2 + x_2^2 + \lambda(x_1 + x_2 - 1)$. Setting $\partial\mathcal{L}/\partial x_1 = 2x_1 + \lambda = 0$ and $\partial\mathcal{L}/\partial x_2 = 2x_2 + \lambda = 0$ gives $x_1 = x_2 = -\lambda/2$. Substituting back:

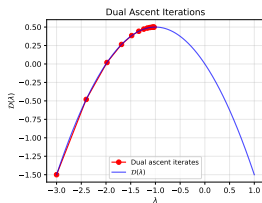
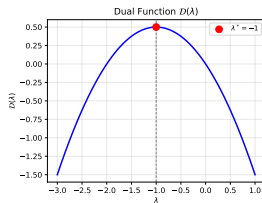
Dual Function

$$\mathcal{D}(\lambda) = -\frac{\lambda^2}{2} - \lambda$$

Setting $\mathcal{D}'(\lambda) = -\lambda - 1 = 0$ gives $\lambda^* = -1$ and $\mathcal{D}(-1) = -1/2 + 1 = 1/2 = p^*$. Strong duality holds.

Dual ascent update rules:

$$x_i^{(k+1)} = -\lambda^{(k)}/2$$



Dual Ascent: Numerical Example II

Worked Example: two steps with $\alpha = 0.5$

Start at $\lambda^{(0)} = 0$.

- Primal step: $x_1 = x_2 = -0/2 = 0$. Violation: $0 + 0 - 1 = -1$.
- Dual step: $\lambda^{(1)} = 0 + 0.5(-1) = -0.5$.
- Primal step: $x_1 = x_2 = -(-0.5)/2 = 0.25$. Violation: $0.25 + 0.25 - 1 = -0.5$.
- Dual step: $\lambda^{(2)} = -0.5 + 0.5(-0.5) = -0.75$.

The sequence converges toward $\lambda^* = -1$, $x_1^* = x_2^* = 1/2$.

Method of Multipliers (Augmented Lagrangian) I

Plain dual ascent can be slow, especially when the dual function is nearly flat (ill-conditioned problem). The **method of multipliers** fixes this by adding a quadratic penalty term directly to the Lagrangian, which sharpens the curvature and accelerates convergence — without requiring $\rho \rightarrow \infty$ as in the pure penalty method.

Augmented Lagrangian

$$\mathcal{L}_\rho(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \boldsymbol{\lambda}^\top \mathbf{h}(\mathbf{x}) + \frac{\rho}{2} \|\mathbf{h}(\mathbf{x})\|_2^2$$

Method of Multipliers (Augmented Lagrangian) II

Notation

- $\mathcal{L}_\rho$ (script L sub rho): the *augmented* Lagrangian, which includes both the standard Lagrange multiplier term and a quadratic penalty.
- $\boldsymbol{\lambda}^\top \mathbf{h}(\mathbf{x})$: the standard Lagrange multiplier term. $\boldsymbol{\lambda}^\top$ (lambda transposed) is the row vector of equality multipliers; multiplying by the column vector $\mathbf{h}(\mathbf{x})$ gives the scalar dot product $\sum_j \lambda_j h_j(\mathbf{x})$.
- $\|\mathbf{h}(\mathbf{x})\|_2^2 = \sum_j h_j(\mathbf{x})^2$: the squared Euclidean norm (denoted $\|\cdot\|_2^2$, “2-norm squared”) of the constraint vector. This is zero only when all equality constraints are satisfied.
- $\rho/2$: the penalty weight. A larger ρ penalises constraint violation more severely.

Method of Multipliers Updates

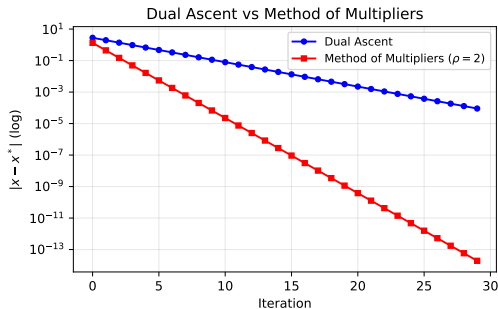
$$\mathbf{x}^{(k+1)} = \arg \min_{\mathbf{x}} \mathcal{L}_\rho(\mathbf{x}, \boldsymbol{\lambda}^{(k)})$$

$$\boldsymbol{\lambda}^{(k+1)} = \boldsymbol{\lambda}^{(k)} + \rho \mathbf{h}(\mathbf{x}^{(k+1)})$$

Method of Multipliers (Augmented Lagrangian) III

Compared to plain dual ascent (step size α), the method of multipliers uses the natural step size ρ for the dual update, and the augmented Lagrangian has stronger curvature in the primal direction. This means convergence holds for any fixed $\rho > 0$ (unlike the pure penalty method, which needs $\rho \rightarrow \infty$ to approach feasibility).

One drawback: the primal update $\min_{\mathbf{x}} \mathcal{L}_{\rho}(\mathbf{x}, \boldsymbol{\lambda}^{(k)})$ must be solved jointly over all components of $\mathbf{x}$, which prevents easy parallelisation. ADMM (the next section) addresses this limitation.



ADMM: Alternating Direction Method of Multipliers I

ADMM combines the benefits of dual ascent (decomposability) and the method of multipliers (convergence for fixed ρ). It does so by splitting the problem into two separate sub-problems that can each be solved independently, then combining results via a shared multiplier.

The problem must be written in the following **canonical ADMM form**:

ADMM Canonical Form

$$\min_{\mathbf{x}_1, \mathbf{x}_2} f_1(\mathbf{x}_1) + f_2(\mathbf{x}_2) \quad \text{subject to} \quad \mathbf{A}_1 \mathbf{x}_1 + \mathbf{A}_2 \mathbf{x}_2 = \mathbf{b}$$

ADMM: Alternating Direction Method of Multipliers II

Notation

- $\mathbf{x}_1 \in \mathbb{R}^{n_1}$, $\mathbf{x}_2 \in \mathbb{R}^{n_2}$: the two blocks of primal variables. Splitting into two blocks is what enables alternating optimisation.
- f_1, f_2 : two convex objective functions, one for each block. They need not have the same form.
- $\mathbf{A}_1 \in \mathbb{R}^{p \times n_1}$, $\mathbf{A}_2 \in \mathbb{R}^{p \times n_2}$: matrices (capital bold A) that couple the two blocks in the linear equality constraint.
- $\mathbf{b} \in \mathbb{R}^p$: the right-hand side (bold lowercase b) of the coupling equality.

ADMM: Alternating Direction Method of Multipliers III

Augmented Lagrangian for ADMM:

$$\mathcal{L}_\rho(\mathbf{x}_1, \mathbf{x}_2, \boldsymbol{\lambda}) = f_1(\mathbf{x}_1) + f_2(\mathbf{x}_2) + \boldsymbol{\lambda}^\top (\mathbf{A}_1 \mathbf{x}_1 + \mathbf{A}_2 \mathbf{x}_2 - \mathbf{b}) + \frac{\rho}{2} \|\mathbf{A}_1 \mathbf{x}_1 + \mathbf{A}_2 \mathbf{x}_2 - \mathbf{b}\|_2^2$$

ADMM Updates

$$\mathbf{x}_1^{(k+1)} = \arg \min_{\mathbf{x}_1} \mathcal{L}_\rho(\mathbf{x}_1, \mathbf{x}_2^{(k)}, \boldsymbol{\lambda}^{(k)}) \quad (\text{update } \mathbf{x}_1, \text{ fix } \mathbf{x}_2)$$

$$\mathbf{x}_2^{(k+1)} = \arg \min_{\mathbf{x}_2} \mathcal{L}_\rho(\mathbf{x}_1^{(k+1)}, \mathbf{x}_2, \boldsymbol{\lambda}^{(k)}) \quad (\text{update } \mathbf{x}_2, \text{ fix new } \mathbf{x}_1)$$

$$\boldsymbol{\lambda}^{(k+1)} = \boldsymbol{\lambda}^{(k)} + \rho (\mathbf{A}_1 \mathbf{x}_1^{(k+1)} + \mathbf{A}_2 \mathbf{x}_2^{(k+1)} - \mathbf{b}) \quad (\text{dual update})$$

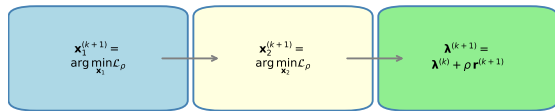
The *alternating direction* is the key feature: rather than minimising jointly over $(\mathbf{x}_1, \mathbf{x}_2)$ together (which may be expensive), we alternate between the two sub-problems. Each sub-problem is typically much

ADMM: Alternating Direction Method of Multipliers IV

cheaper to solve. After updating $\mathbf{x}_1$ and $\mathbf{x}_2$, the dual variable $\boldsymbol{\lambda}$ is updated using the constraint violation $\mathbf{A}_1\mathbf{x}_1 + \mathbf{A}_2\mathbf{x}_2 - \mathbf{b}$, which is the gradient of the augmented Lagrangian with respect to $\boldsymbol{\lambda}$.

ADMM Block Diagram and Convergence I

ADMM Iteration Structure



Stopping criteria: we monitor two residuals to decide when ADMM has converged.

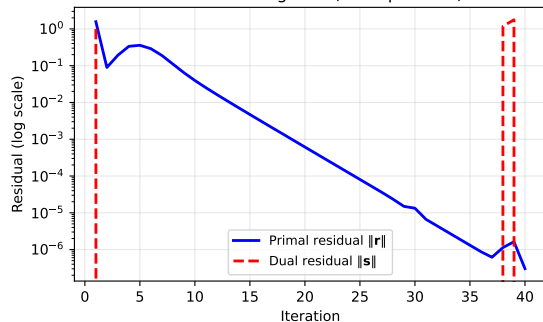
Primal Residual $\mathbf{r}^{(k+1)}$

$$\mathbf{r}^{(k+1)} = \mathbf{A}_1 \mathbf{x}_1^{(k+1)} + \mathbf{A}_2 \mathbf{x}_2^{(k+1)} - \mathbf{b}$$

This measures how well the coupling constraint $\mathbf{A}_1 \mathbf{x}_1 + \mathbf{A}_2 \mathbf{x}_2 = \mathbf{b}$ is satisfied.

Dual Residual $\mathbf{s}^{(k+1)}$

ADMM Convergence (Example 11.4)



Adaptive ρ : the penalty parameter ρ (rho) can be adjusted during the run to balance primal and dual convergence:

- If $\|\mathbf{r}\| \gg \|\mathbf{s}\|$ (primal residual much larger than dual), **increase** ρ . This pushes the

Scaled ADMM Form I

It is often convenient to rescale the dual variable $\boldsymbol{\lambda}$ by dividing by ρ , defining a **scaled dual variable**:

$$\mathbf{u} = \frac{1}{\rho} \boldsymbol{\lambda}$$

Here $\mathbf{u}$ (bold u) is called the scaled dual variable. This simplifies the update formulas considerably.

Substituting $\boldsymbol{\lambda} = \rho \mathbf{u}$ into the augmented Lagrangian, completing the square, and dropping constant terms gives:

$$\mathcal{L}_\rho(\mathbf{x}_1, \mathbf{x}_2, \boldsymbol{\lambda}) = f_1(\mathbf{x}_1) + f_2(\mathbf{x}_2) + \frac{\rho}{2} \|\mathbf{A}_1 \mathbf{x}_1 + \mathbf{A}_2 \mathbf{x}_2 - \mathbf{b} + \mathbf{u}\|_2^2 - \frac{\rho}{2} \|\mathbf{u}\|_2^2$$

Scaled ADMM Form II

Scaled ADMM Updates

$$\mathbf{x}_1^{(k+1)} = \arg \min_{\mathbf{x}_1} f_1(\mathbf{x}_1) + \frac{\rho}{2} \|\mathbf{A}_1 \mathbf{x}_1 - \mathbf{v}^{(k)}\|_2^2, \quad \mathbf{v}^{(k)} = -\mathbf{A}_2 \mathbf{x}_2^{(k)} + \mathbf{b} - \mathbf{u}^{(k)}$$

$$\mathbf{x}_2^{(k+1)} = \arg \min_{\mathbf{x}_2} f_2(\mathbf{x}_2) + \frac{\rho}{2} \|\mathbf{A}_2 \mathbf{x}_2 - \mathbf{w}^{(k)}\|_2^2, \quad \mathbf{w}^{(k)} = -\mathbf{A}_1 \mathbf{x}_1^{(k+1)} + \mathbf{b} - \mathbf{u}^{(k)}$$

$$\mathbf{u}^{(k+1)} = \mathbf{u}^{(k)} + \mathbf{A}_1 \mathbf{x}_1^{(k+1)} + \mathbf{A}_2 \mathbf{x}_2^{(k+1)} - \mathbf{b} = \mathbf{u}^{(0)} + \sum_{j=1}^{k+1} \mathbf{r}^{(j)}$$

Notation

- $\mathbf{v}^{(k)}, \mathbf{w}^{(k)}$: auxiliary target vectors that consolidate the current values of the other block and the scaled dual variable into a single “target” that $\mathbf{x}_1$ (or $\mathbf{x}_2$) is being pulled toward.
- The scaled dual update $\mathbf{u}^{(k+1)} = \mathbf{u}^{(k)} + \mathbf{r}^{(k+1)}$ is simply the **running sum of all past primal residuals** $\mathbf{r}^{(j)} = \mathbf{A}_1 \mathbf{x}_1^{(j)} + \mathbf{A}_2 \mathbf{x}_2^{(j)} - \mathbf{b}$.

Key Insight

Each sub-problem for $\mathbf{x}_1$ and $\mathbf{x}_2$ now has the form: minimise $f_i(\mathbf{x}_i) + \frac{\rho}{2} \|\mathbf{A}_i \mathbf{x}_i - \text{target}\|_2^2$. This is a *proximal minimisation* of f_i with a quadratic proximity term pulling toward the target. Many standard functions (e.g. ℓ_1 norm, indicator of a convex set) have closed-form proximal operators, making each ADMM step very efficient.

Python: ADMM Core Implementation

```
1 import numpy as np
2
3 def admm(f1_prox, f2_prox, A1, A2, b, rho=1.0, gamma=1.5,
4         eps_r=1e-3, eps_s=1e-3, max_iter=200):
5     """
6     ADMM solver for:  $\min f_1(x_1) + f_2(x_2) \quad \text{s.t.} \quad A_1 x_1 + A_2 x_2 = b$ 
7
8     f1_prox(v, rho): proximal operator of f1 --  $\operatorname{argmin} f_1(x) + \rho/2 \|x - v\|^2$ 
9     f2_prox(v, rho): proximal operator of f2
10    """
11    n1 = A1.shape[1]; n2 = A2.shape[1]
12    x1 = np.zeros(n1); x2 = np.zeros(n2); u = np.zeros(len(b))
13
14    for k in range(max_iter):
15        # x1-update: minimise  $f_1(x_1) + \rho/2 \|A_1 x_1 - v\|^2$ 
16        v = -A2 @ x2 + b - u
17        x1 = f1_prox(A1, v, rho)      # application-specific solver
18
19        # x2-update: minimise  $f_2(x_2) + \rho/2 \|A_2 x_2 - w\|^2$ 
20        w = -A1 @ x1 + b - u
21        x2 = f2_prox(A2, w, rho)
22
23        # dual (scaled) variable update
24        r = A1 @ x1 + A2 @ x2 - b      # primal residual
25        s = rho * A1.T @ (x2 - x1)     # dual residual (simplified)
26        u = u + r
27
```

11.5.1 General Constrained Optimisation in ADMM Form I

Any constrained optimisation problem can be reformulated in the ADMM canonical form by introducing an auxiliary variable $\mathbf{x}_2$ and using the *indicator function* of the feasible set as one of the objective terms.

Reformulation: given $\min_{\mathbf{x}} f(\mathbf{x})$ subject to $\mathbf{x} \in \mathcal{X}$ (calligraphic $\mathcal{X}$, some convex constraint set), we write:

$$\min_{\mathbf{x}_1, \mathbf{x}_2} \underbrace{f(\mathbf{x}_1)}_{f_1} + \underbrace{\mathbb{I}_{\mathcal{X}}(\mathbf{x}_2)}_{f_2} \quad \text{subject to} \quad \mathbf{x}_1 - \mathbf{x}_2 = \mathbf{0}$$

where $\mathbb{I}_{\mathcal{X}}(\mathbf{x}_2) = 0$ if $\mathbf{x}_2 \in \mathcal{X}$, and $+\infty$ otherwise. This indicator function encodes the constraint: any solution with $\mathbf{x}_2 \notin \mathcal{X}$ has infinite cost.

11.5.1 General Constrained Optimisation in ADMM Form II

Proximal Minimisation

Each ADMM sub-problem has the form:

$$\arg \min_{\mathbf{x}} f(\mathbf{x}) + \frac{\rho}{2} \|\mathbf{x} - \mathbf{p}\|_2^2$$

where $\mathbf{p}$ (bold p) is the current “target” point. If f is differentiable, a first-order approximation gives the gradient step:

$$\mathbf{x} \approx \mathbf{p} - \frac{1}{\rho} \nabla f(\mathbf{p})$$

This says: start at $\mathbf{p}$, then take a step of size $1/\rho$ in the direction of steepest descent of f .

The $\mathbf{x}_2$ -sub-problem with $f_2 = \mathbb{I}_{\mathcal{X}}$ has an especially simple solution: it is the **projection** of the current $\mathbf{x}_1 + \mathbf{u}$ onto the set $\mathcal{X}$, denoted $\text{proj}_{\mathcal{X}}(\mathbf{x}_1^{(k+1)} + \mathbf{u}^{(k)})$. Projection operators onto common sets (boxes, cones, affine subspaces) often have closed-form expressions.

11.5.2 Alternating Projections I

Goal: find a point $\mathbf{x}$ that belongs to the intersection of two convex sets, $\mathbf{x} \in \mathcal{X}_1 \cap \mathcal{X}_2$. This arises in image reconstruction, signal processing, and constraint satisfaction.

ADMM formulation:

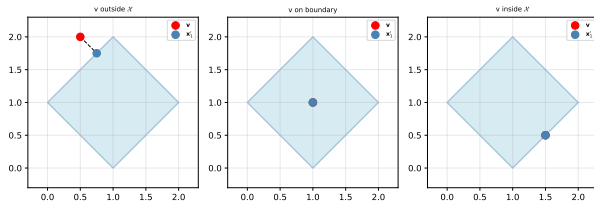
$$\min_{\mathbf{x}_1, \mathbf{x}_2} \mathcal{K}_{\mathcal{X}_1}(\mathbf{x}_1) + \mathcal{K}_{\mathcal{X}_2}(\mathbf{x}_2) \quad \text{subject to} \quad \mathbf{x}_1 - \mathbf{x}_2 = \mathbf{0}$$

Both sub-problems are projections: the $\mathbf{x}_1$ -update projects onto $\mathcal{X}_1$, and the $\mathbf{x}_2$ -update projects onto $\mathcal{X}_2$.

Von Neumann's alternating projections algorithm simply bounces between the two sets: project onto $\mathcal{X}_1$, then onto $\mathcal{X}_2$, and repeat. This converges to the intersection but can be very slow when the two sets meet at a narrow angle (like the inside of a thin wedge).

Dykstra's algorithm (which is equivalent to ADMM here) introduces a correction term $\mathbf{u}$ at each step. This correction acts like a momentum term that remembers the accumulated constraint violations and steers the iterates more directly toward the intersection. Dykstra

Proximal Minimization: Projection onto $\mathcal{X}$



11.5.3 Least Absolute Deviations (LAD) I

Ordinary least squares minimises $\|\mathbf{Ax} - \mathbf{b}\|_2^2$, which is sensitive to outliers because large errors are squared. **Least Absolute Deviations (LAD)** instead minimises $\|\mathbf{Ax} - \mathbf{b}\|_1$ (the sum of absolute residuals), which is much more robust.

LAD Problem

$$\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_1 = \sum_{i=1}^m |(\mathbf{Ax} - \mathbf{b})_i|$$

ADMM formulation: introduce $\mathbf{x}_2 = \mathbf{Ax}_1 - \mathbf{b}$:

$$\min_{\mathbf{x}_1, \mathbf{x}_2} f_1(\mathbf{x}_1) + \|\mathbf{x}_2\|_1 \quad \text{subject to} \quad \mathbf{Ax}_1 - \mathbf{x}_2 = \mathbf{b}$$

The key to solving the $\mathbf{x}_2$ -update is the **soft thresholding operator**, which is the proximal operator of the ℓ_1 (L1) norm:

11.5.3 Least Absolute Deviations (LAD) II

Soft Thresholding Operator S_κ

$$(S_\kappa(\mathbf{x}))_i = \begin{cases} x_i + \kappa & x_i < -\kappa \\ 0 & |x_i| \leq \kappa \\ x_i - \kappa & x_i > \kappa \end{cases} = \text{sign}(x_i) \max(|x_i| - \kappa, 0)$$

Notation

- κ (kappa, pronounced “cap-a”): the threshold level. Values of $|x_i|$ smaller than κ are shrunk to exactly zero; larger values are shifted toward zero by κ .
- $\text{sign}(x_i)$: returns $+1$ if $x_i > 0$, -1 if $x_i < 0$, and 0 if $x_i = 0$.

11.5.3 Least Absolute Deviations (LAD) III

How to Read This

Soft thresholding shrinks every element of a vector toward zero by the amount κ , and sets it to exactly zero if it was already smaller than κ in magnitude. It is the “soft” version of hard thresholding (which would set small values to zero and leave large values unchanged). Soft thresholding is the key operation that produces sparse solutions.

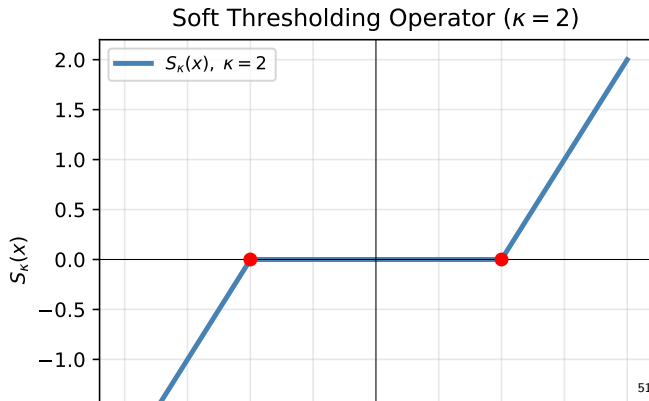
LAD ADMM updates:

$$\mathbf{x}'_1 = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top (\mathbf{b} + \mathbf{x}_2 - \mathbf{u})$$

$$\mathbf{x}'_2 = S_{1/\rho}(\mathbf{A}\mathbf{x}'_1 - \mathbf{b} + \mathbf{u})$$

$$\mathbf{u}' = \mathbf{u} + \mathbf{A}\mathbf{x}'_1 - \mathbf{b} - \mathbf{x}'_2$$

The Cholesky factorisation of $\mathbf{A}^\top \mathbf{A}$ is precomputed once, making each iteration $O(mn)$ instead of $O(n^3)$.



11.5.4 Basis Pursuit (Split Bregman) I

Basis pursuit finds the sparsest solution to an underdetermined linear system. In compressed sensing and signal recovery, we observe a short measurement vector $\mathbf{b}$ and know a measurement matrix $\mathbf{A}$, and we want to recover a high-dimensional sparse signal $\mathbf{x}$:

Basis Pursuit Problem

$$\min_{\mathbf{x}} \|\mathbf{x}\|_1 \quad \text{subject to} \quad \mathbf{Ax} = \mathbf{b}$$

The ℓ_1 norm $\|\mathbf{x}\|_1 = \sum_i |x_i|$ promotes sparsity: solutions with many zero components are preferred because they have smaller ℓ_1 norm (for the same constraint satisfaction) than dense solutions.

ADMM reformulation: split $\mathbf{x}$ into two copies $\mathbf{x}_1 = \mathbf{x}_2$, putting the ℓ_1 penalty on $\mathbf{x}_2$ and the equality constraint on $\mathbf{x}_1$:

11.5.4 Basis Pursuit (Split Bregman) II

ADMM Updates for Basis Pursuit

$$\begin{aligned}\mathbf{x}'_1 &= (\mathbf{I} - \mathbf{A}^\top (\mathbf{A}\mathbf{A}^\top)^{-1} \mathbf{A})(\mathbf{x}_2 - \mathbf{u}) + \mathbf{A}^\top (\mathbf{A}\mathbf{A}^\top)^{-1} \mathbf{b} \\ \mathbf{x}'_2 &= S_{1/\rho}(\mathbf{x}'_1 + \mathbf{u}) \quad \mathbf{u}' = \mathbf{u} + \mathbf{x}'_1 - \mathbf{x}'_2\end{aligned}$$

Notation

- $\mathbf{I} - \mathbf{A}^\top (\mathbf{A}\mathbf{A}^\top)^{-1} \mathbf{A}$: the *orthogonal projector* onto the null space of $\mathbf{A}$ (the set of all $\mathbf{x}$ satisfying $\mathbf{A}\mathbf{x} = \mathbf{0}$).
- $\mathbf{A}^\top (\mathbf{A}\mathbf{A}^\top)^{-1} \mathbf{b}$: the minimum-norm particular solution to $\mathbf{A}\mathbf{x} = \mathbf{b}$ (the pseudo-inverse solution).
- The $\mathbf{x}'_1$ update therefore takes any starting point $\mathbf{x}_2 - \mathbf{u}$, projects it onto the feasible affine subspace $\{\mathbf{x} : \mathbf{A}\mathbf{x} = \mathbf{b}\}$.

Both matrices $\mathbf{I} - \mathbf{A}^\top (\mathbf{A}\mathbf{A}^\top)^{-1} \mathbf{A}$ and $\mathbf{A}^\top (\mathbf{A}\mathbf{A}^\top)^{-1} \mathbf{b}$ can be precomputed once (costing $O(p^3)$ for the matrix inverse of the $p \times p$ Gram matrix $\mathbf{A}\mathbf{A}^\top$). Each subsequent iteration then costs only one

11.5.4 Basis Pursuit (Split Bregman) III

matrix-vector product ($O(np)$) plus soft thresholding ($O(n)$). This method is also known as the **split Bregman** method.

11.5.5 Lasso: L_1 -Regularised Regression I

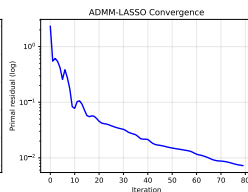
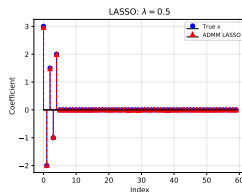
The Lasso (Least Absolute Shrinkage and Selection Operator) balances fitting a linear model to data against producing a sparse coefficient vector:

Lasso Problem

$$\min_{\mathbf{x}} \frac{1}{2} \|\mathbf{Ax} - \mathbf{b}\|_2^2 + \lambda \|\mathbf{x}\|_1$$

Notation

- $\frac{1}{2} \|\mathbf{Ax} - \mathbf{b}\|_2^2$: the least-squares fit term, measuring how well $\mathbf{Ax}$ approximates $\mathbf{b}$. The factor $1/2$ is included for convenience so the gradient equals $\mathbf{A}^\top (\mathbf{Ax} - \mathbf{b})$ without a factor of 2.
- $\lambda \|\mathbf{x}\|_1$ (lambda times L1 norm): the regularisation term. $\lambda \geq 0$ (lambda, “lam-da”) controls the sparsity trade-off. Larger λ pushes more coefficients to exactly zero.



11.5.5 Lasso: L_1 -Regularised Regression II

Lasso ADMM Updates

$$\mathbf{x}'_1 = (\mathbf{A}^\top \mathbf{A} + \rho \mathbf{I})^{-1} (\mathbf{A}^\top \mathbf{b} + \rho(\mathbf{x}_2 - \mathbf{u}))$$

$$\mathbf{x}'_2 = S_{\lambda/\rho}(\mathbf{x}'_1 + \mathbf{u})$$

$$\mathbf{u}' = \mathbf{u} + \mathbf{x}'_1 - \mathbf{x}'_2$$

Notation

- $\mathbf{A}^\top \mathbf{A} + \rho \mathbf{I}$: adding $\rho \mathbf{I}$ (rho times the identity matrix $\mathbf{I}$) to $\mathbf{A}^\top \mathbf{A}$ makes it strictly positive definite for any $\rho > 0$, guaranteeing invertibility even when $\mathbf{A}$ is rank-deficient.
- $S_{\lambda/\rho}$: soft thresholding with threshold λ/ρ . As ρ decreases, the threshold λ/ρ increases, producing sparser solutions.

The matrix $\mathbf{A}^\top \mathbf{A} + \rho \mathbf{I}$ can be factorised (e.g. via Cholesky) once at the start, making the $\mathbf{x}_1$ -update a single triangular solve at each iteration ($O(n^2)$ after factorisation).

Python: ADMM-Lasso Implementation (Algorithm 11.7)

```
1 import numpy as np
2
3 def lasso_admm(A, b, lam, x1=None, x2=None, rho=1.0,
4               gamma=2.0, eps=1e-3, max_iter=100):
5     """
6     Solve  $\min 0.5 \|Ax - b\|^2 + \text{lam} \|x\|_1$ 
7     via ADMM (Algorithm 11.7 in Python).
8     """
9     n = A.shape[1]
10    if x1 is None: x1 = np.zeros(n)
11    if x2 is None: x2 = np.zeros(n)
12    u = np.zeros(n)
13
14    AtA = A.T @ A
15    Atb = A.T @ b
16
17    def soft(v, kappa):
18        return np.sign(v) * np.maximum(np.abs(v) - kappa, 0)
19
20    for k in range(max_iter):
21        # x1-update: linear system solve (warm-startable)
22        x1 = np.linalg.solve(AtA + rho * np.eye(n),
23                             Atb + rho * (x2 - u))
24        # x2-update: soft thresholding
25        x2 = soft(x1 + u, lam / rho)
26        # scaled dual update
27        u = u + x1 - x2
```

Huber Loss and Other ADMM Applications I

The **Huber loss** is a hybrid between squared (ℓ_2) and absolute (ℓ_1) error that is both smooth near zero and robust to outliers far from zero:

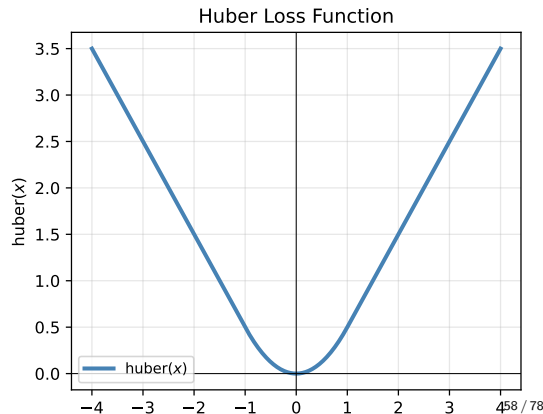
Huber Loss

$$\text{huber}(x) = \begin{cases} \frac{1}{2}x^2 & |x| \leq 1 \\ |x| - \frac{1}{2} & |x| > 1 \end{cases}$$

How to Read This

For small residuals ($|x| \leq 1$): the Huber loss grows quadratically, just like squared error, giving smooth gradients near the optimum. For large residuals ($|x| > 1$): the loss grows only linearly, like the absolute value, so outliers contribute at most linearly and do not dominate the fit. This makes Huber regression robust without sacrificing smoothness.

The $\mathbf{x}_2$ -update for Huber regression in ADMM is:



Huber Loss and Other ADMM Applications II

Summary of ADMM application patterns:

Application	$f_2(\mathbf{x}_2)$	$\mathbf{x}_2$ -update
Constrained opt.	$\mathbb{I}_{\mathcal{X}}$ (indicator)	Project onto $\mathcal{X}$
Basis pursuit	$\ \cdot\ _1$ (ℓ_1 norm)	Soft threshold
Lasso	$\lambda\ \cdot\ _1$	Soft threshold (scaled)
LAD regression	$\ \cdot\ _1$	Soft threshold
Huber regression	huber loss	Blend of linear + soft

The pattern is always the same: one sub-problem handles the smooth objective (solved by a linear system), the other handles the non-smooth or constrained term (solved by a proximal/projection step).

Consensus Optimisation I

In many large-scale problems, the objective is the sum of k separate functions:

$$\min_{\mathbf{x}} f_1(\mathbf{x}) + f_2(\mathbf{x}) + \cdots + f_k(\mathbf{x})$$

This arises naturally when k different agents (processors, data shards, or physical subsystems) each have their own local objective f_i , but all must agree on a common decision variable $\mathbf{x}$. The challenge is to optimise the sum without collecting all data in one place.

Consensus reformulation: introduce a local copy $\mathbf{x}_1^{(i)}$ for each agent, and a global variable $\mathbf{x}_2$ that all local copies must agree with:

ADMM Consensus Form

$$\min_{\mathbf{x}_1^{(1)}, \dots, \mathbf{x}_1^{(k)}, \mathbf{x}_2} \sum_{i=1}^k f_i(\mathbf{x}_1^{(i)}) \quad \text{subject to} \quad \mathbf{x}_1^{(i)} = \mathbf{x}_2 \quad \forall i = 1, \dots, k$$

Notation

- $\mathbf{x}_1^{(i)}$ (x sub 1 superscript i): agent i 's local copy of the decision variable.
- $\mathbf{x}_2$: the global consensus variable that all agents must agree with.
- $\forall i$ (for all i): the constraint must hold for every agent $i = 1, \dots, k$.

Consensus Optimisation III

Consensus ADMM Updates

Step 1 — Local updates (all k agents update in parallel):

$$(\mathbf{x}_1^{(i)})' = \arg \min_{\mathbf{x}_1^{(i)}} f_i(\mathbf{x}_1^{(i)}) + (\boldsymbol{\lambda}^{(i)})^\top (\mathbf{x}_1^{(i)} - \mathbf{x}_2) + \frac{\rho}{2} \|\mathbf{x}_1^{(i)} - \mathbf{x}_2\|_2^2$$

Step 2 — Consensus update (global averaging):

$$\mathbf{x}_2' = \frac{1}{k} \sum_{i=1}^k \left[(\mathbf{x}_1^{(i)})' + \frac{1}{\rho} \boldsymbol{\lambda}^{(i)} \right]$$

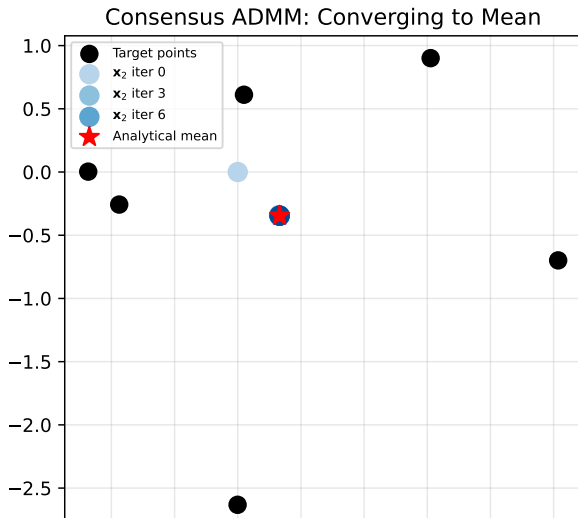
Step 3 — Dual updates (one per agent):

$$(\boldsymbol{\lambda}^{(i)})' = \boldsymbol{\lambda}^{(i)} + \rho \left[(\mathbf{x}_1^{(i)})' - \mathbf{x}_2' \right]$$

Key Insight

Step 1 runs entirely in parallel: each agent i solves its own sub-problem involving only f_i , $\lambda^{(i)}$ (its own multiplier), and the current global consensus $\mathbf{x}_2$. The sub-problem details of f_i do not need to be shared with anyone. Only the local solutions $(\mathbf{x}_1^{(i)})'$ and the multipliers $\lambda^{(i)}$ need to be communicated to compute the average in Step 2. This makes consensus ADMM suitable for *federated learning* and privacy-preserving distributed optimisation.

Consensus ADMM: Convergence to Mean I



How consensus ADMM converges:

Initially, each agent's local copy $\mathbf{x}_1^{(i)}$ focuses on minimising its own f_i without caring about the others. This gives a diverse set of local optima. The dual variables $\lambda^{(i)}$ (lambda for agent i) then penalise divergence between $\mathbf{x}_1^{(i)}$ and the global mean $\mathbf{x}_2$. Over successive iterations, the dual variables grow until the disagreement cost outweighs the individual objective savings, and all local copies converge to a common consensus value.

Key properties:

- Each $\mathbf{x}_1^{(i)}$ -update can run on a separate processor with access only to f_i .
- Communication per iteration: each agent sends $\mathbf{x}_1^{(i)}$ and $\lambda^{(i)}$ to the coordinator, which broadcasts $\mathbf{x}_2$.

Python: Consensus ADMM (Algorithm 11.8)

```
1 import numpy as np
2 from concurrent.futures import ThreadPoolExecutor
3
4 def consensus_admm(objectives, x1_inits, x2_init,
5                    rho=1.0, gamma=2.0, eps_r=1e-3, eps_s=1e-3,
6                    max_iter=100):
7     """
8     Consensus ADMM: min sum_i f_i(x) (f_i in objectives list)
9     Each f_i is a callable returning (value, gradient).
10    Uses scipy.optimize.minimize for each x1^(i) update.
11    """
12    from scipy.optimize import minimize as sci_min
13    k = len(objectives)
14    x1s = [np.array(xi, dtype=float) for xi in x1_inits]
15    x2 = np.array(x2_init, dtype=float)
16    lams = [np.zeros_like(x2) for _ in range(k)]
17
18    x1_bar_old = np.mean(x1s, axis=0)
19
20    for it in range(max_iter):
21        # Parallel x1^(i) updates
22        def update_i(args):
23            i, f_i, x1, lam = args
24            def obj(x):
25                val = f_i(x)
26                penalty = lam @ (x - x2) + 0.5*rho*np.sum((x - x2)**2)
27                return val + penalty
```

11.6.2 General Consensus Optimisation I

In the basic consensus formulation, every agent's local variable $\mathbf{x}_1^{(i)}$ has the same dimension as the global variable $\mathbf{x}_2$. In many real problems, each agent only cares about a *subset* of the global variables. General consensus handles this case.

General Consensus Formulation

$$\min_{\mathbf{x}_1^{(1)}, \dots, \mathbf{x}_1^{(k)}, \mathbf{x}_2} \sum_{i=1}^k f_i(\mathbf{x}_1^{(i)}) \quad \text{subject to} \quad \mathbf{x}_1^{(i)} = \mathbf{x}_2^{(i)} \quad \forall i$$

where $\mathbf{x}_2^{(i)}$ denotes the sub-vector of $\mathbf{x}_2$ containing only the components relevant to agent i .

11.6.2 General Consensus Optimisation II

General Consensus Updates

$$(\mathbf{x}_1^{(i)})' = \arg \min_{\mathbf{x}_1^{(i)}} f_i(\mathbf{x}_1^{(i)}) + (\boldsymbol{\lambda}^{(i)})^\top (\mathbf{x}_1^{(i)} - \mathbf{x}_2^{(i)}) + \frac{\rho}{2} \|\mathbf{x}_1^{(i)} - \mathbf{x}_2^{(i)}\|_2^2$$

$$(\mathbf{x}_2)_g' = \frac{1}{n_g} \sum_{i: g \in l_i} \left[(\mathbf{x}_1^{(i)})' + \frac{1}{\rho} \boldsymbol{\lambda}^{(i)} \right]_g$$

$$(\boldsymbol{\lambda}^{(i)})' = \boldsymbol{\lambda}^{(i)} + \rho [(\mathbf{x}_1^{(i)})' - (\mathbf{x}_2^{(i)})']$$

11.6.2 General Consensus Optimisation III

Notation

- $(\mathbf{x}_2)_g$: component g of the global variable $\mathbf{x}_2$.
- n_g : the number of agents whose local variable includes component g .
- The $\mathbf{x}_2$ -update averages only over those agents who “participate” in component g (denoted $g \in I_i$ meaning component g is in agent i 's index set I_i).

The key advantage is that each agent's local sub-problem is smaller (lower-dimensional) than the full problem, and agents that do not share any components do not need to communicate at all.

Example 11.7: Bakery Placement as General Consensus I

This example shows how general consensus handles a real logistics problem where different agents care about different subsets of the global variables.

The setup: place 3 bakeries at positions $\mathbf{b}^{(1)}$, $\mathbf{b}^{(2)}$, $\mathbf{b}^{(3)}$ (2D coordinates each) to minimise total delivery distances to 6 establishments:

- 2 cafes at $\mathbf{e}^{(1)} = [0, 0]$, $\mathbf{e}^{(2)} = [0.5, 0]$ — supplied by bakery 1 (croissants)
- 2 brunch spots at $\mathbf{e}^{(3)} = [3, 2]$, $\mathbf{e}^{(4)} = [0.5, 3]$ — supplied by bakeries 1 and 2
- 2 restaurants at $\mathbf{e}^{(5)} = [3.5, -0.5]$, $\mathbf{e}^{(6)} = [2.5, 0.7]$ — supplied by bakeries 2 and 3 (baguettes, dinner rolls)

Global variable: $\mathbf{x}_2 = [b_1^{(1)}, b_2^{(1)}, b_1^{(2)}, b_2^{(2)}, b_1^{(3)}, b_2^{(3)}]^\top$ (the x - and y -coordinates of all three bakeries).

Each establishment acts as an agent with a local variable containing only the coordinates of its bakery suppliers. For example, cafe 1 has local variable $\mathbf{x}_1^{(1)} = [b_1^{(1)}, b_2^{(1)}]$ (only bakery 1 coordinates), while brunch spot 1 has $\mathbf{x}_1^{(3)} = [b_1^{(1)}, b_2^{(1)}, b_1^{(2)}, b_2^{(2)}]$ (bakeries 1 and 2). No agent needs information about bakeries they do not use, enabling true distributed computation.

Example 11.7: Bakery Placement as General Consensus II

Key Insight

In this example, bakery 3's position $[b_1^{(3)}, b_2^{(3)}]$ only appears in the local variables of restaurants (agents 5 and 6). The cafes and brunch spots need not know where bakery 3 is. General consensus ADMM automatically handles this partial overlap through the component-wise averaging in the $\mathbf{x}_2$ update.

Chapter 11 Summary I

This chapter developed the theory and practice of duality, showing how every constrained minimisation problem has a companion dual maximisation problem, and how exploiting this structure leads to powerful algorithms.

Theoretical foundations:

- The **dual function** $\mathcal{D}(\mu, \lambda)$ is always concave, making maximisation tractable.
- **Weak duality** ($d^* \leq p^*$) always holds: the dual value is a guaranteed lower bound.
- **Strong duality** ($d^* = p^*$) holds for convex problems satisfying Slater's condition (strictly feasible interior point exists).
- **Complementary slackness**: at optimality, $\mu_i^* g_i(\mathbf{x}^*) = 0$ for all i — either the multiplier or the slack must be zero.

Primal-dual interior-point:

- Simultaneously updates $\mathbf{x}$ and μ .
- Uses Newton step on the residual vector.

Dual ascent family:

- **Dual ascent**: alternate between minimising the Lagrangian and ascending the dual. Converges but slow.
- **Method of multipliers**: add quadratic penalty to Lagrangian; converges for fixed ρ .
- **ADMM**: split the primal into two blocks; each sub-problem is cheaper. Enables parallelism.
- **Consensus ADMM**: k agents optimise in parallel; only averages are communicated; privacy-preserving.
- **General consensus**: agents have overlapping but non-identical variable sets; enables

Chapter 11 Summary II

Method	Primal update	Dual update
Dual ascent	Lagrangian min	Gradient step
Method of multipliers	Aug. Lagrangian min	Multiplier step
ADMM	Two alternating min	Multiplier step
Consensus ADMM	k parallel min	Mean + multiplier

The Central Takeaway

Duality transforms hard constrained minimisation into concave maximisation, provides lower bounds for verification, and enables the efficient decomposition of large problems into small parallel sub-problems. ADMM is a workhorse of modern large-scale optimisation precisely because it exploits this dual structure.

Selected Exercises I

Exercise 11.1. Show that complementary slackness $\mu_i^* g_i(\mathbf{x}^*) = 0$ holds for all i whenever the duality gap is zero.

Proof sketch: A zero gap means $d^* = p^* = f(\mathbf{x}^*)$. By definition:

$$d^* = \mathcal{D}(\boldsymbol{\mu}^*, \boldsymbol{\lambda}^*) \leq \mathcal{L}(\mathbf{x}^*, \boldsymbol{\mu}^*, \boldsymbol{\lambda}^*) = f(\mathbf{x}^*) + \sum_i \mu_i^* g_i(\mathbf{x}^*) \leq f(\mathbf{x}^*) = p^*$$

Since $d^* = p^*$, all inequalities are equalities. In particular $\sum_i \mu_i^* g_i(\mathbf{x}^*) = 0$. Since each $\mu_i^* \geq 0$ and $g_i(\mathbf{x}^*) \leq 0$, every term is non-positive and their sum is zero, so each term is exactly zero: $\mu_i^* g_i(\mathbf{x}^*) = 0$. This is complementary slackness.

Exercise 11.2. Is the dual of the dual equal to the primal?

Answer: Not in general. Let dd^* denote the optimal value of the dual of the dual. From weak duality applied twice: $dd^* \leq d^* \leq p^*$. If a duality gap exists ($d^* < p^*$), then $dd^* \neq p^*$. For convex problems with strong duality, however, $dd^* = d^* = p^*$.

Selected Exercises II

Exercise 11.3. Derive the ADMM $\mathbf{x}_1$ -update when $f_1(\mathbf{x}_1) = \frac{1}{2}\mathbf{x}_1^\top \mathbf{Q}\mathbf{x}_1 + \mathbf{q}^\top \mathbf{x}_1 + q$ with $\mathbf{Q}$ (capital bold Q) positive semidefinite.

Solution: The $\mathbf{x}_1$ -sub-problem is:

$$\min_{\mathbf{x}_1} \frac{1}{2}\mathbf{x}_1^\top \mathbf{Q}\mathbf{x}_1 + \mathbf{q}^\top \mathbf{x}_1 + \lambda^\top \mathbf{A}_1\mathbf{x}_1 + \frac{\rho}{2}\|\mathbf{A}_1\mathbf{x}_1 + \mathbf{A}_2\mathbf{x}_2 - \mathbf{b}\|_2^2$$

Setting the gradient with respect to $\mathbf{x}_1$ to zero gives the normal equations:

$$(\mathbf{Q} + \rho\mathbf{A}_1^\top \mathbf{A}_1)\mathbf{x}'_1 = \rho\mathbf{A}_1^\top (\mathbf{A}_2\mathbf{x}_2 - \mathbf{b}) - \mathbf{q} - \lambda$$

This is a positive definite linear system when $\mathbf{Q} + \rho\mathbf{A}_1^\top \mathbf{A}_1$ is positive definite (guaranteed for $\rho > 0$ when $\mathbf{A}_1$ has full column rank, or always if $\mathbf{Q}$ is positive definite).

Exercise 11.6. Simplify the consensus $\mathbf{x}_2$ -update.

Selected Exercises III

Solution: Let $\bar{\mathbf{x}}_1 = (1/k) \sum_i (\mathbf{x}_1^{(i)})'$ (the mean of all local updates) and $\bar{\lambda} = (1/k) \sum_i \lambda^{(i)}$ (the mean multiplier). Then the consensus update simplifies to:

$$\mathbf{x}'_2 = \bar{\mathbf{x}}_1 + \frac{1}{\rho} \bar{\lambda}$$

This removes the need for an explicit $\mathbf{x}_2$ variable: the global consensus is simply the mean of local solutions plus a dual correction.

Algorithm Complexity Summary I

Algorithm	Per-iteration cost	Key parameter
Primal-dual IP	$O((n + m)^3)$ Newton solve	ρ increased each iter
Dual ascent	$O(\text{cost of } \min_{\mathbf{x}} \mathcal{L})$	step size α (alpha)
Method of multipliers	$O(\text{cost of aug. Lag. min})$	$\rho > 0$ fixed
ADMM (basic)	$O(n^3)$ per sub-problem	ρ, γ (gamma)
ADMM-Lasso	$O(n^3)$ (one-time Chol.) $+ O(n^2)$ each	ρ, λ
Consensus ADMM	$O(k \cdot \text{sub-problem cost}),$ parallel	ρ, γ

Here n is the number of primal variables, m is the number of inequality constraints, k is the number of agents in consensus, $O(\cdot)$ (big-O notation) means “proportional to at most this cost as problem size grows,” and “Chol.” means the Cholesky factorisation of the coefficient matrix (done once and reused each iteration).

Practical guidelines for running ADMM:

Algorithm Complexity Summary II

- **Adaptive ρ :** monitor the ratio $\|\mathbf{r}\|/\|\mathbf{s}\|$ at each iteration. Scale ρ up (e.g. by a factor $\gamma = 2$) when $\|\mathbf{r}\| \gg \|\mathbf{s}\|$; scale down when the reverse holds. This keeps the two residuals balanced and avoids slow convergence.
- **Initialisation:** start with a small ρ (e.g. 1) and let adaptive scaling do the rest. A warm-started ρ from a previous problem instance can also help.
- **Precompute factorisations:** for problems like Lasso and basis pursuit, factor $\mathbf{A}^\top \mathbf{A} + \rho \mathbf{I}$ or $\mathbf{A} \mathbf{A}^\top$ once before the loop. Each iteration then costs a triangular solve instead of a full linear solve.
- **Convergence speed:** ADMM typically shows linear convergence (the residual decreases by a constant factor each iteration). Near the solution, a few Newton-like refinement steps can dramatically improve accuracy.

References

- Kochenderfer & Wheeler, *Algorithms for Optimization*, 2nd ed., MIT Press, 2026.
- S. Boyd & L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004.
- S. J. Wright, *Primal-Dual Interior-Point Methods*, SIAM, 1997.
- S. Boyd, N. Parikh, E. Chu, B. Peleato, J. Eckstein, "Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers," *Foundations and Trends in Machine Learning*, vol. 3, no. 1, 2011.
- R. Tibshirani, "Regression Shrinkage and Selection via the Lasso," *JRSS-B*, vol. 58, 1996.
- T. Goldstein & S. Osher, "The Split Bregman Method for L_1 -Regularized Problems," *SIAM Journal on Imaging Sciences*, vol. 2, 2009.